```
for convo <- convos,
    message <- convo.messages,
    String.starts_with?(message.from, "+1") or message.from == "Reader",
    message.status == "received" do
  {message.from, message.body, length(convo.messages)}
end

# => [{"Reader", "text", 3}, {"+1-222-333-4444", nil, 2}]
```

This example iterates over each convo, iterates over each message, filters to only include messages that start with "+1" or "Reader", filters to only include received messages, and then constructs a result based on the message and conversation. That's a lot of stuff in a small amount of code!

This code would be more verbose if we wrote it using Enum functions, so comprehensions provide a clean syntax for data enumeration. But it's totally okay to use Enum. Start with the technique you feel most comfortable with.

Wrapping Up

A clean data layer makes it easier to work with, maintain, and extend a software application. Elixir provides a whole host of data types, but lists and maps are among the most important as you model an application's data.

Lists hold any data in your application. They are easy to work with, but their performance characteristics mean that you need to be a bit cautious about how you use them. Keyword lists are a special type of list that are most commonly used to pass around options in your application.

Maps are the heart of an application's data layer. They are simply containers that hold key-value pairs, but their performance and ease of use make them the most common data type. Structs are a special type of map with additional features like specified fields, required fields, and default values. At their core, structs are just maps, so they are easy to work with.

You can create self-contained data structures by putting the functions that modify or read your data structures into the same module that defines the struct. Elixir provides several options for how you can iterate or reduce a data structure. The Enum module and for comprehensions are usually the best options for working with a complex data structure.

Up next, we're going to cover one of the most exciting features in the Elixir language—pattern matching.